ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

NAME:P DILLI BABU

REG NO:192472284

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 20-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

GitHub Repository Link: https://github.com/dillibabu1122/DSA0115--C-/tree/main/AT3

Q1 – Constructors and Constructor Overloading

1. Design and Implementation

The Book class uses a default constructor for an empty record, a title-only constructor for a donated book, and a parameterised constructor for a complete record. Constructor delegation sends all construction through one validation point. The number of copies is never allowed to remain negative.

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class Book {
    string title;
    int copies;
    string author;

    static int validCopies(int n) {
        if (n < 0) throw invalid_argument("Copies cannot be negative");
        return n;
    }

public:
    Book() : Book("", 0, "Unknown") {}

    explicit Book(const string& t) : Book(t, 1, "Unknown") {}

    Book(const string& t, int n, const string& a)
        : title(t), copies(validCopies(n)), author(a) {}

    void display() const {
        cout << "Title: " << title << ", Copies: " << copies
             << ", Author: " << author << '\n';
    }
```

```
};

int main() {
    Book empty;
    Book donated("C++ Basics");
    Book complete("Data Structures", 5, "Mark Allen");

    empty.display();
    donated.display();
    complete.display();

    try {
        Book invalid("Invalid Book", -2, "Unknown");
    } catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << '\n';
    }
    return 0;
}
```

Sample output:

```
Title: , Copies: 0, Author: Unknown
Title: C++ Basics, Copies: 1, Author: Unknown
Title: Data Structures, Copies: 5, Author: Mark Allen
Error: Copies cannot be negative
```

2. Design and Analysis

- The compiler supplies a default constructor only when the class declares no constructor at all. Once a user-declared constructor exists, the compiler does not automatically add a no-argument default constructor.

- A member initializer list directly initializes members. It is required for const members, reference members and members without a default constructor; it also avoids default construction followed by assignment.
- Members are initialized in the order in which they are declared in the class, not the order written in the initializer list.
- Constructor delegation lets one constructor call another constructor of the same class, reducing duplicated validation and initialization.

Object	Constructor used	Final state
empty	Book() → Book("", 0, "Unknown")	title="", copies=0, author="Unknown"
donated	Book(title) → full constructor	title="C++ Basics", copies=1, author="Unknown"
complete	Book(title, copies, author)	title="Data Structures", copies=5, author="Mark Allen"

3. Experimental Investigation

- Remove Book() and try Book books[3];. The array declaration fails because each element needs a default constructor.
- To observe -Wreorder, declare members in one order but list their initializers in a different order. Compile with -Wall -Wextra; the compiler warns that initialization follows declaration order.
- Try a negative copy count. The constructor throws invalid_argument and the object is not successfully created.

4. explicit and Alternative Design

The title-only constructor is marked explicit to prevent an implicit conversion from string to Book. For example, a function taking Book cannot silently accept a string when the constructor is explicit. Without explicit, such a conversion may compile.

```
// With explicit:
void show(Book b) {}
// show(string("C++ Basics")); // rejected: implicit conversion is disabled
// show(Book("C++ Basics"));   // valid direct construction
```

Alternative: one constructor with default arguments could be Book(string title = "", int copies = 0, string author = "Unknown"). Object creation remains O(1) time and O(1) object space. Both approaches can validate copies in one constructor body, but overloaded constructors communicate the three intended creation cases more clearly. A single default-argument constructor can become ambiguous or less expressive as options grow.

Q2 – Copy Constructors: Shallow and Deep Copies

1. Why Shallow Copy Fails

A compiler-generated copy of a raw pointer copies only the address. Two Prescription objects then point to the same medicine array. Changing one changes the other, and both destructors may delete the same memory, causing a double-free error.

Shallow-copy memory view: p.medicines ──┐ and q.medicines ──┴──> one shared array. Deep-copy memory view: p.medicines ──> Array A; q.medicines ──> independent Array B.

2. Deep-Copy Implementation (Rule of Three)

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new string[count] : nullptr) {}

    Prescription(const Prescription& other)
        : n(other.n), medicines(other.n > 0 ? new string[other.n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
    }
```

```
    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        string* fresh = other.n > 0 ? new string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription() {
        delete[] medicines;
    }

    void set(int i, const string& m) { medicines[i] = m; }
    string get(int i) const { return medicines[i]; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

3. Rule of Three and Copy Situations

- Rule of Three: if a class needs a user-defined destructor, copy constructor, or copy assignment operator, it usually needs all three.
- The copy constructor is used when a new object is initialized from another object, when an object is passed by value, and when an object is returned by value (subject to copy elision).
- Passing by const reference does not make a copy. Passing by value normally requires a copy or move, although compiler optimizations may remove some copies.
- Self-assignment p = p is safe because the assignment operator checks this == &other before changing resources.

4. Experiments and Complexity

Add a print statement to the copy constructor to count copies. Call a function once with Prescription by value and once with const Prescription&; compare the counts. Test self-assignment.

Number of medicines n	Deep-copy time	Additional storage
10	O(n)	O(n)
1,000	O(n)	O(n)
100,000	O(n)	O(n)

A shallow pointer copy is O(1) time and O(1) extra storage, but it is unsafe because ownership is shared. Deep copy is O(n) because every medicine name is copied. With std::vector<string>, the compiler-generated copy performs an independent element copy automatically; this is the Rule of Zero and avoids writing the three special member functions.

Run both versions with -fsanitize=address. The shallow version may report a double-free/use-after-free when both objects release the shared pointer. Record the exact report produced by your own compiler; it can vary by environment.

5. Complete-the-Code TODO Answers

- TODO 1: Define Prescription(const Prescription& other), allocate a new array of other.n strings, then copy each element.
- TODO 2: Check self-assignment, allocate and populate a new array, delete the old array, assign the new pointer and size, and return *this.
- TODO 3: Use delete[] medicines; in the destructor.

Q3 – Dynamic Constructors and Destructors

1. Dynamic Two-Dimensional Seat Map

```
#include <iostream>
using namespace std;

class Event {
    int rows, seats;
    bool** booked;

public:
    Event(int r, int s) : rows(r), seats(s), booked(nullptr) {
        if (rows < 0 || seats < 0) {
            rows = seats = 0;
        }
        if (rows > 0) {
            booked = new bool*[rows];
            int made = 0;
            try {
                for (; made < rows; ++made) {
                    booked[made] = new bool[seats]{};
                }
            } catch (...) {
                while (made > 0) delete[] booked[--made];
                delete[] booked;
                booked = nullptr;
```

```
            throw;
        }
    }
    cout << "Event created: " << rows << " x " << seats << '\n';
}

bool book(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) return false;
    if (booked[r][s]) return false;
    booked[r][s] = true;
    return true;
}

bool cancel(int r, int s) {
    if (r < 0 || r >= rows || s < 0 || s >= seats) return false;
    if (!booked[r][s]) return false;
    booked[r][s] = false;
    return true;
}

~Event() {
    int count = 0;
    for (int r = 0; r < rows; ++r) {
        for (int s = 0; s < seats; ++s)
            if (booked[r][s]) ++count;
        delete[] booked[r];
    }
    delete[] booked;
    cout << "Event closed; booked seats: " << count << '\n';
}
};

int main() {
```

```
    Event local(2, 3);
    local.book(0, 1);
    {
        Event blockEvent(1, 2);
        blockEvent.book(0, 0);
    }

    Event* heapEvent = new Event(2, 2);
    heapEvent->book(1, 1);
    delete heapEvent;
    return 0;
}
```

Illustrative output:

```
Event created: 2 x 3
Event created: 1 x 2
Event closed; booked seats: 1
Event created: 2 x 2
Event closed; booked seats: 1
Event closed; booked seats: 1
```

2. Object Lifetime and Memory

- A local object is constructed when execution reaches its declaration and destroyed when its scope ends.
- A block-scope object is destroyed at the closing brace of that block.
- A dynamically allocated object is constructed by new and remains alive until delete is called.
- Forgetting delete leaks the object and its allocated memory. Using delete instead of delete[] for an array is undefined behavior.
- A destructor has no parameters and no return type. It cannot be overloaded because there is only one destruction operation for an object.

Memory layout: booked points to an array of row pointers. Each row pointer points to a separate bool[seats] allocation. Therefore the number of allocations is rows + 1.

3. RAII and Alternative Concepts

RAII ties resource release to object lifetime. When an Event leaves scope, its destructor automatically releases the seat map, reducing the chance of forgotten cleanup. A production implementation should prefer std::vector or smart pointers to simplify exception safety.

Representation	Number of allocations	Access time	Extra row-pointer storage
Separate row allocations	rows + 1	O(1)	O(rows)
One contiguous block	1	O(1)	None
vector<vector<char>>	Implementation-managed; typically one per row plus outer storage	O(1)	O(rows) metadata

For a single block, allocate rows * seats elements and access element r,s using data[r * seats + s]. All representations provide O(1) indexed access. A zero-row event should allocate no row map and should safely report zero booked seats.

For 10×20, 100×200 and 1,000×2,000 maps, the raw bool payload is respectively 200, 20,000 and 2,000,000 bytes if bool occupies one byte on the implementation. Actual allocation overhead is implementation-dependent; measure it rather than assuming it.

4. Predict the Output – Logger

```
#include <iostream>
#include <string>
using namespace std;

struct Logger {
    string n;
```

```
    Logger(const string& s) : n(s) { cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { cout << "copy" << n << ' '; }
    ~Logger() { cout << '-' << n << ' '; }
};

void use(Logger) { cout << "use "; }

int main() {
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    cout << "end ";
}
```

Expected output (C++17):

```
+A +B -B copyAc use -Ac +P -P end -A
```

Explanation: a is created first and survives until main ends. b is created inside the block and destroyed at its closing brace. Passing a by value creates a parameter copy named Ac; it is destroyed when use returns. The dynamically allocated P object is created by new and destroyed by delete. Finally, a is destroyed as main exits.

5. Alternative Seat-Map Design

A contiguous block uses one allocation and the index formula r * seats + s. A vector<vector<char>> provides convenient automatic memory management but generally stores row containers separately. All options have O(1) access; the contiguous representation has fewer allocations and better locality.

Q4 – Overloading Arithmetic and Stream Operators

1. Money Class

```
#include <iomanip>
#include <iostream>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }

    bool operator==(const Money& other) const {
        return paise == other.paise;
    }

    friend ostream& operator<<(ostream& out, const Money& m) {
        long long value = m.paise;
        if (value < 0) {
            out << "-Rs ";
            value = -value;
        } else {
            out << "Rs ";
        }
        out << value / 100 << '.'
```

```
            << setw(2) << setfill('0') << value % 100
            << setfill(' ');
        return out;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';
    Money x(0, 75), y(0, 50);
    cout << x + y << '\n';
}
```

Expected output:

```
Rs 200.00
true
Rs 1.25
```

2. Operator Design

- The + operator returns a new Money object and does not change either operand.
- The += operator should modify the left operand and return Money& (*this), allowing chained operations without an unnecessary copy.
- The == and < operators can be members because they compare the current object with another Money value.
- The << operator must be a non-member because its left operand is ostream, not Money. A friend declaration gives it access to private paise.
- The expression a + b is translated conceptually to a.operator+(b) when the member overload is selected.

For a billing system, adding, subtracting and comparing money values make sense. Multiplying one Money value by another generally does not: the result would have squared currency units. Multiplying Money by a quantity or tax rate can be meaningful with a carefully defined type and rounding policy.

Negative subtraction results need a policy. This sample permits signed amounts and prints a minus sign. Another design could reject negative balances or return an error, depending on billing requirements.

3. Alternative Concept and Complexity

The operators could be implemented as non-member friend functions or replaced by named functions such as add(a,b) and equals(a,b). All operations are O(1) time and O(1) additional space. A non-member operator can support symmetric expressions such as 100 + m as well as m + 100 when suitable overloads are provided. A member overload naturally has the class object as its left operand.

For the required experiment, compare a double-based implementation with the integer-paise implementation over 1,000,000 additions. Integer paise avoids many binary floating-point representation surprises. Record actual elapsed time from your own run.

Q5 – Increment, Comparison and Subscript Operators

1. Time and Playlist Program

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Time {
    int seconds;
public:
    explicit Time(int s = 0) : seconds(s < 0 ? 0 : s) {}
    Time& operator++() { ++seconds; return *this; }       // prefix
    Time operator++(int) { Time old = *this; ++seconds; return old; } // postfix
    bool operator==(const Time& t) const { return seconds == t.seconds; }
    bool operator<(const Time& t) const { return seconds < t.seconds; }
```

```
    int value() const { return seconds; }
};

class Playlist {
    vector<string> tracks;
public:
    void add(const string& name) { tracks.push_back(name); }
    string& operator[](size_t i) {
        if (i >= tracks.size()) throw out_of_range("Playlist index out of range");
        return tracks[i];
    }
    const string& operator[](size_t i) const {
        if (i >= tracks.size()) throw out_of_range("Playlist index out of range");
        return tracks[i];
    }
    size_t size() const { return tracks.size(); }
};

int main() {
    Time t(10);
    Time a = t++;
    Time b = ++t;
    cout << a.value() << ' ' << b.value() << ' ' << t.value() << '\n';

    Playlist list;
    list.add("Track One");
    list.add("Track Two");
    cout << list[0] << '\n';
    try {
        cout << list[2] << '\n';
    } catch (const out_of_range& e) {
        cout << e.what() << '\n';
    }
```

```
}
```

Expected output:

```
10 12 12
Track One
Playlist index out of range
```

2. Explanation

- Prefix ++ is operator++() with no dummy parameter. Postfix ++ is operator++(int); the unused int distinguishes it.
- Postfix returns the old value, so it must preserve a copy before incrementing. Prefix increments and returns *this by reference.
- operator[] needs a non-const version returning string& so callers can modify an element, and a const version returning const string& for read-only playlists.
- An out-of-range index should be handled safely. This solution throws std::out_of_range rather than accessing invalid memory.

For a = t++ and b = ++t starting at 10: a receives 10, then t becomes 11; prefix increments t to 12 and b receives 12. Therefore a=10, b=12 and t=12.

3. Complexity and Experiments

Prefix and postfix increment are O(1) for this small Time class. Postfix uses one temporary object; prefix does not. Playlist access is O(1). A checked access adds one comparison, still O(1). std::vector::at(i) throws std::out_of_range; unchecked operator[] does not perform a bounds check.

Measure 10,000,000 prefix and postfix increments at -O0 and -O2, and compare checked and unchecked access. Compiler optimization, hardware and benchmark design affect timings, so enter only measured results. Test negative indexes carefully: a negative int converted to size_t becomes a very large positive value and will be rejected by the range check. Test 59:59 rollover separately if Time is represented as hours, minutes and seconds.

Q6 – Rules of Operator Overloading

1. Text Class with Dynamic Memory

```
#include <cstring>
#include <iostream>
using namespace std;

class Text {
    char* data;
public:
    Text(const char* s = "") {
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    Text(const Text& other) {
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);
    }

    Text& operator=(const Text& other) {
        if (this != &other) {
            char* fresh = new char[strlen(other.data) + 1];
            strcpy(fresh, other.data);
            delete[] data;
            data = fresh;
        }
        return *this;
    }

    ~Text() { delete[] data; }

    Text operator+(const Text& other) const {
```

```
        size_t n = strlen(data) + strlen(other.data);
        char* joined = new char[n + 1];
        strcpy(joined, data);
        strcat(joined, other.data);
        Text result(joined);
        delete[] joined;
        return result;
    }

    bool operator==(const Text& other) const {
        return strcmp(data, other.data) == 0;
    }

    bool operator!=(const Text& other) const {
        return !(*this == other);
    }

    friend ostream& operator<<(ostream& out, const Text& t) {
        return out << t.data;
    }
};

int main() {
    Text a("Hello "), b("World");
    Text c = a + b;
    cout << c << '\n';
    cout << boolalpha << (c == Text("Hello World")) << '\n';
    c = c; // self-assignment test
    Text empty;
    cout << "Empty=[" << empty << "]\n";
}
```

Expected output:

```
Hello World
true
Empty=[]
```

2. Operator Overloading Rules

- The operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- Overloading cannot change an operator's precedence, associativity or number of operands (arity).
- At least one operand must be a user-defined type; an overload cannot redefine an operator for two built-in types only.
- operator= must be a member function.
- A friend is useful when a non-member operator needs private access, such as stream insertion where ostream is the left operand.

The Rule of Three applies because Text owns a dynamically allocated character array: it defines a destructor, copy constructor and copy assignment operator. The assignment operator allocates the new buffer before deleting the old one and checks self-assignment.

3. Copies, Complexity and Alternative

For a + b + c, the left-to-right grouping is (a + b) + c. The first concatenation copies the characters of a and b into a new result; the second copies that intermediate result and c. If the lengths are n1, n2 and n3, the character work is proportional to (n1+n2) + (n1+n2+n3), i.e. O(2n1+2n2+n3), which is O(n1+n2+n3) asymptotically but performs an intermediate allocation.

Using std::string is normally preferable because it already manages memory and provides tested operations. An alternative operator+(Text a, const Text& b) { a += b; return a; } can reuse the by-value left-hand copy, but requires a correct += implementation. For many concatenations, reserving final capacity or using append can reduce repeated allocations.

Compare 100,000 concatenations against std::string and run under AddressSanitizer. Record actual timing and sanitizer output. Test empty strings, long messages and self-assignment.

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Interpret the Problem

An impedance is represented by a real part and an imaginary part: Z = a + jb. The class must support construction, arithmetic, equality, rectangular printing, magnitude calculation and safe division.

- Data: two double values, real and imag.
- Operations: addition, subtraction, multiplication, division, equality, magnitude and stream output.
- Constructors: default (0 + 0j), real-only (a + 0j), and real-plus-imaginary (a + bj).
- Measurable requirements: each arithmetic operation returns the mathematically correct result; division by zero is detected; magnitude equals sqrt(a^2+b^2); output displays both components.

Stage 2 – Design and Formulae

Operator	Formula
Addition	(a+jb)+(c+jd) = (a+c)+j(b+d)
Subtraction	(a+jb)-(c+jd) = (a-c)+j(b-d)
Multiplication	(ac-bd)+j(ad+bc)
Division	((ac+bd)+j(bc-ad))/(c^2+d^2)
Magnitude	sqrt(a^2+b^2)
Equality	Compare real and imaginary components within a tolerance

Arithmetic operators can be members. Stream insertion is a friend non-member because ostream is the left operand. Division checks whether the denominator magnitude is zero before calculating.

Stage 3 – Implementation and Verification

```
#include <cmath>
#include <iostream>
#include <stdexcept>
using namespace std;

class Complex {
    double real, imag;
public:
```

```
    Complex() : real(0), imag(0) {}
    Complex(double r) : real(r), imag(0) {}
    Complex(double r, double i) : real(r), imag(i) {}

    Complex operator+(const Complex& z) const {
       return Complex(real + z.real, imag + z.imag);
    }
    Complex operator-(const Complex& z) const {
       return Complex(real - z.real, imag - z.imag);
    }
    Complex operator*(const Complex& z) const {
       return Complex(real*z.real - imag*z.imag,
                      real*z.imag + imag*z.real);
    }
    Complex operator/(const Complex& z) const {
       double d = z.real*z.real + z.imag*z.imag;
       if (d < 1e-24) throw domain_error("Division by zero impedance");
       return Complex((real*z.real + imag*z.imag)/d,
                      (imag*z.real - real*z.imag)/d);
    }
    bool operator==(const Complex& z) const {
       const double eps = 1e-9;
       return fabs(real-z.real) < eps && fabs(imag-z.imag) < eps;
    }
    double magnitude() const { return sqrt(real*real + imag*imag); }

    friend ostream& operator<<(ostream& out, const Complex& z) {
       out << z.real;
       if (z.imag >= 0) out << " + j" << z.imag;
       else out << " - j" << -z.imag;
       return out;
    }
};
```

```
int main() {
    const double w = 100.0;
    const double R = 10.0, L = 0.1, C = 0.001;
    Complex zr(R, 0);
    Complex zl(0, w*L);
    Complex zc(0, -1.0/(w*C));

    Complex series = zr + zl + zc;
    Complex parallel = (zr * zl) / (zr + zl);

    cout << "Series impedance: " << series << '\n';
    cout << "Series magnitude: " << series.magnitude() << '\n';
    cout << "Parallel impedance: " << parallel << '\n';

    try {
        Complex zero;
        Complex invalid = zr / zero;
        cout << invalid << '\n';
    } catch (const domain_error& e) {
        cout << "Error: " << e.what() << '\n';
    }
}
```

Verification by hand for the chosen sample values: ω=100 rad/s, R=10 Ω, L=0.1 H and C=0.001 F. Inductive reactance is ωL=10 Ω, so ZL=j10. Capacitive reactance is 1/(ωC)=10 Ω, so ZC=−j10. The series impedance is therefore 10+j0 Ω and its magnitude is 10 Ω. The parallel result in this particular program combines R and L only, using (ZR×ZL)/(ZR+ZL); the division-by-zero test is expected to print the error message.

Stage 4 – Defend the Design

- The default constructor represents zero impedance; the real-only constructor represents a resistor; the two-component constructor represents general impedance.

- The arithmetic operators directly express circuit equations and make the program readable.
- std::complex<double> provides standard, tested complex arithmetic and is preferred in production code unless a custom class is needed for teaching, special validation, units or formatting.
- Both implementations store two double values and each basic arithmetic operation is O(1) time and O(1) space.
- A remaining limitation is floating-point precision: exact equality is unsuitable for most measured engineering values, so tolerance-based comparison is used.

For a complete parallel RLC calculation, construct the total impedance of each branch and combine them using the parallel formula. The sample demonstrates a series RLC calculation and a two-branch parallel calculation. Record your own compiler output and compare it with hand calculations.

Conclusion:

Constructors establish valid object states, destructors release owned resources, and carefully designed operator overloads make user-defined types intuitive. Deep copying or standard library containers prevent ownership errors, while testing and complexity analysis help defend the design.